

AI编程是否会取代全部程序员？看看业界的实际情况

知乎 · 2026-09-19

自从25年末26年初AI编程的效果大大提升之后，程序员的编码工作很大一部分甚至基本上都被AI替代了。这一现象在程序员群体里引起了恐慌和绝望情绪。与此同时，AI公司利用这一领域的进展，无限夸大AI的功能，号称一两年之内白领统统下岗。各种自媒体纷纷跟上，传播恐慌，制造焦虑。

关于这个问题，我们有必要认真地考察一下。AI编程是否能代替全部程序员？如果不能，那实际替代率有多少？

实际上，我几个月前写的一个回答（[AI第一剑，先斩程序员，程序员要大量失业了吗？](#)）中，就给出了自己的判断：AI无法替代全部人工，端到端的替代率不高于30%。

当以AI为核心的智能机器系统+大规模进入生产之后，人工编码工作基本上会消失。当然，架构复杂或者性能要求特别高的核心代码开发有可能还是需要人工高度介入，但大量普通的代码开发，将全面被AI代替。具体会代替多少人呢？按照我的实际使用效果，以及其他人在知乎上的反馈来看，大约能提效20%到50%不等，如果AI coding能正常落地（现在还在行业的大规模试验中），按照纯开发人员占比60%估计，大约削减12%到30%的程序员岗位。

AI coding大规模落地已经至少半年了，尤其是在中美，基本上大厂都要求员工使用AI Agent开发。那么，效果到底如何？

根据麦肯锡+的报告[1]，“会用AI”的程序员，可将生产率提高大约16%~30%左右，与我之前的判断大体一致。



最近的一项研究[2]也表明，虽然AI编程让代码生成速度大幅度提升，但是提现到最终的产品发布上，效率近提升30%。这又与上面麦卡锡的研究以及我之前的推测一致。

用于软件开发的生成式AI工具+能力进步神速，短短几年间便从简单的代码自动补全演变为完全自主的编码智能体（coding agents）。这些能力的提升能否转化为现实世界中任务层面的生产力增长？即便能够转化，任务层面的效率提升又是否意味着最终产出的增加？这些问题对于理解AI的经济影响至关重要。通过针对开发者的匹配事件研究，我们发现，随着AI编码工具

的迭代升级，其在任务层面带来的生产力提升效应也愈发显著：自动补全功能使每周代码提交量增加了约30%，**同步型智能体+**（sync agents）将这一累积效应提升至约180%，而**异步型智能体+**（async agents）则将其提升至约240%。然而，随着层级向上推移至生产环节，这些增益效应便急剧减弱：尽管同步型智能体使代码行数增加了958%、拉取请求（pull requests）增加了86%，但最终发布的版本数量仅增长了20%。**GitHub+**上的总体编码活动也呈现出同样的规律。



来源：<https://mitsloan.mit.edu/ideas-made-to-matter/ai-boosts-worker-productivity-does-translate-to-final-outputs>

不仅各种研究正在证实AI编程局限性，而且实际使用AI Agent的开发人员和管理者对此更是有直接的感受。程序员对AI编程的态度，已经从最初的震惊中清醒过来，通过实际的生产应用，逐渐认识到AI的局限性。比如，**Cal Newport**（乔治城大学计算机科学教授）的一篇博客文章[3]就谈到了硅谷的一名资深程序员这半年来的转变。他年初用**Claude Code+**编程后，生产率提高几倍，大呼“really, really good”，在经历了AI代码引起的两次线上事故、差点丢掉工作后，对AI编程感到幻灭，开始手写核心代码，AI编程只用于写测试用例和脚本之类的代码。这名开发者认为，**古法编程+**虽然慢，但是可靠，质量有保障。作者表示，这种情况并不是孤立的，有不少行业内的工程师都在表达类似的看法。

AI编程问题在哪？一位程序员在博客[4]中谈到他半年来检视AI代码的感想。他认为，AI Agent就像一名非常优秀的新员工，代码写得看起来很好，但因为缺乏领域背景知识，总是对各种约束条件、边界和异常场景考虑不周，还容易出安全问题。实际上，最近一篇论文《**约束衰减+**：LLM 智能体在后端代码生成中的脆弱性》[5]中也谈到了这个问题：

我们的研究揭示了一种“约束衰减”现象：随着结构性要求的增加，智能体的性能会出现显著下降。从基准任务到完全指定要求的任务，表现较好的配置在断言通过率上平均下降了 30 个百分点，而一些表现较弱的配置甚至降至接近零的水平。针对框架的**敏感性分析+**揭示了显著的性能差异：智能体在极简且显式的框架（如 **Flask+**）中表现良好，但在高度依赖约定的环境（如 FastAPI、**Django+**）中，平均性能则大幅下滑。最后，错误分析表明，数据层缺陷（例如错误的查询构建和 **ORM+** 运行时异常）是导致这些问题的主要根源。这项研究强调，如何同时满足功能性和结构性要求，对于编程智能体而言仍然是一项关键且尚未解决的挑战。

印度企业的一位CTO+在最近的一篇文章里也提到[6]，AI写的代码容易在实际生产运行中出问题，是因为“在大型系统中，代码必须符合架构模式、依赖链、业务规则、合规要求、安全管控及发布流程。若 AI 生成的代码忽视这些约束，会将工作重心从开发转移到后续的修复与整改上，进而导致代码审查周期延长、测试覆盖不足、发现安全隐患、集成失败以及生产环境缺陷等问题。”有过实际AI编程经验的人，就能理解这些问题确实是广泛存在的。

AI代码写得快，但是由于如上所述的这些问题，导致开发人员必须仔细检视AI生成的代码，才能将其合入代码仓。这就引出两个问题。第一，代码检视速度显著低于代码生成速度，使得AI的提效大打折扣。第二，新程序员缺少动手写代码的机会，能力无法提升，等老一批程序员被裁员或者退休后，最终会导致代码检视速度进一步变慢，甚至检视不出问题。

更有甚者，有些企业对AI编程寄予厚望，大幅压缩开发周期，使得程序员根本没时间认真检视代码。对于开发人员来说，企业也不是自己的，上级要求快，那就简单检视甚至不检视即合并。如果上线后出问题，大不了走人。无论是检视疲劳，还是检视不充分，最终的结果是进入生产环境的代码里潜藏着一大堆BUG和代码坏味道之类的问题，而负责维护这些软件的人，实际上对代码是不熟悉的。这一切技术债务造成了所谓的“可维护性差距”（The Maintainability Gap）[7]。

AI 辅助编程持续迅猛发展，最具积极性的开发者产出的代码量也创下了历史新高。然而，基于四年代码变更数据的分析显示，代码的可维护性指标正在下滑：跨文件函数调用（体现了代码复用）减少了 35%；与 2022 年水平相比，重构中的代码行迁移减少了 70%，针对长期遗留代码的维护工作也减少了 74%。

与此同时，我们观察到一些令人担忧的现象呈上升趋势：单次提交内的“复制/粘贴”行为增加了 41%，代码块重复增加了 81%，掩盖错误的编码结构增加了 47%，两周内的代码流失率（churn+）也增加了 15%。虽然代码产出确实大幅提升，但随之累积的技术债务同样不容忽视；这些债务主要集中在那些尚未意识到某些编码模式会危及代码库长期可维护性的开发者身上。

关于技术债务，论文《Debt Behind the AI Boom》[8]通过大规模的研究表明，“代码异味+是最常见的问题类型，占比高达 89.3%”，“各类 AI 编程助手产生的提交中，均有超过 15% 至少引入了一个问题。更重要的是，在 AI 引入的问题中，有 22.7% 在代码仓库的最新版本中依然存在”。“这些发现表明，AI 生成的代码可能会给实际软件项目带来长期的维护成本，同时也凸显了在 AI 辅助开发中加强质量保证的必要性。”



而论文《Faster Code, Deeper Debt?》[9]则指出,“LLM往往会加剧传统形式的技术债务(特别是代码、设计和文档方面的债务),同时也会引入LLM特有的新型债务。值得注意的是,我们识别出了一种‘快速集成债务’:即为了追求速度而牺牲质量,导致快速生成的代码引发连锁反应,进而产生治理债务并增加长期维护成本。其他新兴债务类别还包括提示词(prompt)债务、伦理债务、数据债务和溯源债务,这些反映了LLM应用所带来的独特挑战。”

AI编程不仅导致了技术债务的积累，还会引入其他债务。计算机科学家Margaret-Anne Storey+在发表的论文《From Technical Debt to Cognitive and Intent Debt》[10]提出，在技术债务之外，“随着人工智能生成代码的速度超过了团队理解代码的速度，两种常被低估的“债务”形式正在累积：一是“认知债务”（cognitive debt），即团队内部共识的流失；二是‘意图债务’（intent debt），即缺乏开发者和人工智能智能体安全处理代码所需的外部化逻辑依据。”翻译成大白话就是，AI编程的全面使用可能导致整个团队没人清楚自己在干啥。目前，业界的关注点仍然聚焦在技术债务上，这些更深层次的问题还没有完全暴露出来。

看到这里，我觉得读者朋友心中肯定已经有答案了。我之前在很多回答（如：[你认为AI编程未来真的会取代程序员吗？](#)和[如何看待微软 AI 负责人预测称，「未来 18 个月内，多数白领工作将被 AI 彻底替代」？](#)）中都在反复讲一个观点：AI会深刻改变白领的工作方式，但是AI不可能把全部程序员淘汰掉，更不可能让全部白领都下岗。实际情况证明了我的观点是正确的。

随着AI代替全部白领这种神话的破灭，资本市场已经开始出现泡沫破裂的迹象。我之前按照软件开发提效30%的假设推断出AI泡沫的确存在，而且规模大于互联网泡沫（参考：《[AI泡沫到底有多](#)
[大？AI泡沫危机是否会远超08年金融危机？](#)》）。最近，AI大模型企业的CEO放出来的AI毁灭人类论，不过是灭亡前的最后疯狂。

参考

1. ^The AI revolution in software development <https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/the-ai-revolution-in-software-development>
2. ^Writing Code vs. Shipping Code: Productivity Effects Across Generations of AI Coding Tools. <https://download.ssrn.com/2026/9/6/6843118.pdf?response-content-disposition=inline&X-Amz-Security-Token=IQoJb3JpZ2luX2VjEIR%2F%2F%2F%2F%2F%2F%2F%2F%2FwEaCXVzLWVhc3QtMSJHMEUCIA4KlxaTLIbYG8l%2B4GGUGWJIOPlbXefVltiMEDrjOT7UAiEAxlaBFnksn8BvTmtJugDcWKlimIcsG%2BFtjqO5fZmGv3ggvQUIUxAEGgwzMDgONzUzMDEyNTciDL6mjhl5>

IYItgnLyySqaBXzwW0%2BLuUe1Po7ACCP5mwJ4tKLHkJ7VxCmWECyy1j%2B18zQrjPZ9d9oVhTucj3AeO8TSgT%2FdBwQKaRdz%2FPW5X9g8ck8Fzs jYPV5ADNNYitwvTqRPH0tmwIvdANXIo2MmE%2Fq0tUxp8r7dYnLKml71nao%2FoObCOuBU5bFDtp8z8L6peJLcFA1xYV%2FeYRQkwubgHG%2FWCr4xxKPH0z%2FAqzs2Fy%2FCZV11TA38%2BRZ7CfsFrB8LbWKaXFZtjW2hGat4LtLubJMYIsDjew4S8JxMJAh7AeyBkh7DLufKHdNwBDozx%2FlaJycLzA7KKd6EnmSVjIa8HNACI4sibNpVZcfiFtbAQvQmoiqlB%2B2RHggWcbS9ZA8dRigETQmzcZSV3GJkKiYDOGIOPiBI59uItj6R54orFsestnCMONvNvRXEtK40HOZT3aTFC1D6D4dcQZCLB5VlvtBF6dwzTpxT4SKagfjfenKolISfu5Jzf6Vb2SZJfU9xZxJWESIVQGru%2BkahJyAni%2BNmo7ZzX%2B0cbgjUbGn5J4q47zi41KMrhsGDamp1ALeQtDwTmTYhs70fRNF6%2Fq6ix52SBYcWt8QuhUtagYDmm6%2FaGG0VkASEbmW8hoOI2chBQEjBDUegB9NGgplbFydPSyX9WI2Ds0GZMcJT1ZHcjYia6tbSSwKF4jICpBFPXyMf33i0bhQWRjNXunDOBE3UFyI6Fzm%2BG%2BBE0mDrNVwYATYcOGXVr9v2KbTR6HjWEZCUHgeKLevjTBrdJzNXu1%2FgkIWJSt7tIWPPgGCMe8vElex9QMfDi945ep58TXzu61kXHeZA2rk56Rc1UEEc71bnV%2Bs2BibiYEWiSiXfd4TNFEMehOwdLpS0pRQNN5MJ4zu6sjTGE1S6L1j%2FTC6zbfVBjqxAb6xSZFcMCJuzZhj7V%2B46YKbBPIGM5omjl6jenIlgv2E%2BHrp9t1l%2B6Dt5A8oetYaA8S%2FCI8nfSGx8bUk8%2FRzbediBWTeAYfyta39CDTTGZzqWGX4Zg2FYmVySCcUFJL%2B%2Fi8AyosppJMVPwCI26CSJquHPD54%2BLssBLEFZNBGEXoeayHvn1JN4NCfFjioMOP%2B8czxCNN1WRklxigrdJae4Ew52iInuVz%2FDmizuSzgvtww9A%3D%3D&X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Date=20260919T025315Z&X-Amz-SignedHeaders=host&X-Amz-Expires=300&X-Amz-Credential=ASIAUPUUPRWEYZTNYLVN%2F20260919%2Fus-east-1%2Fs3%2Faws4_request&X-Amz-Signature=4ba6ead523ab5e286d4d945a3af8b9fbf80ada4d1e8f200c179fede48d0732b3&abstractId=6843118

3. ^On AI Coding and Its Discontents <https://calnewport.com/on-ai-coding-and-its-discontents/>
4. ^I Spent 6 Months Reviewing AI Code. Here's the Ugly Truth https://medium.com/@haseeb_sohail/i-spent-6-months-reviewing-ai-code-heres-the-ugly-truth-b81fe33e0b25
5. ^Constraint Decay: The Fragility of LLM Agents in Backend Code Generation <http://arxiv.org/pdf/2605.06445>
6. ^The Productivity Illusion at the Heart of Enterprise AI Coding <https://cacm.acm.org/blogcacm/the-productivity-illusion-at-the-heart-of-enterprise-ai-coding/>
7. ^The Maintainability Gap: AI Code Quality in 2026 https://www.gitclear.com/the_ai_code_quality_maintainability_gap
8. ^Debt Behind the AI Boom: A Large-Scale Empirical Study of AI-Generated Code in the Wild <https://arxiv.org/abs/2603.28592>

9. ^Faster Code, Deeper Debt? A Multivocal Literature Review on Technical Debt and Its Early Signs in LLM-Assisted Software Development <https://arxiv.org/abs/2606.14796>
10. ^From Technical Debt to Cognitive and Intent Debt: Rethinking Software Health in the Age of AI <https://arxiv.org/abs/2603.22106>

[1]

<https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/the-ai-revolution-in-software-development>

Writing Code vs. Shipping Code: Productivity Effects Across Generations of AI Coding Tools.

[3]

On AI Coding and Its Discontents

[4]

I Spent 6 Months Reviewing AI Code. Here's the Ugly Truth

https://medium.com/@haseeb_sohail/i-spent-6-months-reviewing-ai-code-heres-the-ugly-truth-b81fe33e0b25

[5]

Constraint Decay: The Fragility of LLM Agents in Backend Code Generation

<https://arxiv.org/pdf/2605.06445>

[6]

The Productivity Illusion at the Heart of Enterprise AI Coding

<https://cacm.acm.org/blogcacm/the-productivity-illusion-at-the-heart-of-enterprise-ai-coding/>

[7]

The Maintainability Gap: AI Code Quality in 2026

https://www.gitclear.com/the_ai_code_quality_maintainability_gap

[8]

Debt Behind the AI Boom: A Large-Scale Empirical Study of AI-Generated Code in the Wild

<https://arxiv.org/abs/2603.28592>

[9]

Faster Code, Deeper Debt? A Multivocal Literature Review on Technical Debt and Its Early Signs in LLM-Assisted Software Development

<https://arxiv.org/abs/2606.14796>

[10]

From Technical Debt to Cognitive and Intent Debt: Rethinking Software Health in the Age of AI

<https://arxiv.org/abs/2603.22106>

本文原发表于知乎（中文）：<https://zhuanlan.zhihu.com/p/2084561490930045240>

作者：最最遥远的路 • [知乎主页](#)